

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

GUI
Botones y oyentes

Dr. Luciano H. Tamargo
<http://cs.uns.edu.ar/~lt>
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

TEMARIO

- Repaso.
- Construcción de una GUI.
- Organizador de diagramado.
- Botones.

REPASO

- Una **interface gráfica de usuario (GUI)** es un medio que permite que una persona se comuniquen y controle un sistema a través de ventanas, botones, menús, etc.
- Una GUI se construye a partir de una **colección de componentes** con una **representación gráfica** y capacidad para **percibir eventos** generados por las acciones del usuario.

REPASO

- La construcción de una GUI está fuertemente relacionada con los conceptos de:
 - Encapsulamiento
 - Herencia
 - Polimorfismo
- Y con los criterios de:
 - Reusabilidad
 - Extensibilidad

TEMARIO

- Repaso.
- Construcción de una GUI.
- Organizador de diagramado.
- Botones.

CONSTRUCCIÓN DE UNA GUI

- **Diseñar** la interfaz de acuerdo a las especificaciones.
- **Implementar** la interfaz usando las facilidades provistas por el lenguaje.

DISEÑO DE UNA GUI

- Definir las componentes.
- Definir el diagramado de las componentes contenedoras.
- Decidir cómo reacciona cada componente ante las acciones realizadas por el usuario.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

7

IMPLEMENTACIÓN DE UNA GUI

- Crear un objeto gráfico para cada componente de la GUI e **insertarlo** en otras componentes contenedoras.
- Establecer** los valores de los atributos de los objetos gráficos.
- Definir el comportamiento** de las componentes reactivas en respuesta a las acciones del usuario.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

8

OBJETOS GRÁFICOS Y CLASES GRÁFICAS

- Un objeto gráfico es una instancia de una **clase gráfica**.
- Una clase gráfica define algunos **atributos gráficos** y brinda **servicios gráficos**.
- Una clase gráfica puede usarse para:
 - Crear objetos gráficos asociados a las componentes de la interfaz.
 - Definir clases más específicas a partir de las cuales se crearán componentes.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

9

TEMARIO

- Repaso.
- Construcción de una GUI.
- Organizador de diagramado.
- Botones.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

10

ORGANIZADOR DE DIAGRAMADO

- El **organizador de diagramado** es un atributo de todos los objetos gráficos contenedores que determina como se distribuyen las componentes contenidas.
- Algunas de las clases provistas para crear organizadores son **BorderLayout**, **FlowLayout**, **GridLayout**.
- En la clase práctica trabajarán con distintos diagramados.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

11

ORGANIZADOR DE DIAGRAMADO

- FlowLayout**: Distribuye los componentes uno al lado del otro comenzando en la parte superior.
- BorderLayout**: Divide el contenedor en cinco regiones: NORTH, SOUTH, EAST, WEST y CENTER, admite un único componente por región.
- GridLayout**: Divide el contenedor en una grilla de n filas por m columnas, con todas las celdas de igual tamaño.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
10011
0 0
1 1
1
```

12

TEMARIO

- Repaso.
- Construcción de una GUI.
- Organizador de diagramado.
- Botones.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

13

BOTONES

- Un **botón** es una componente **reactiva**, puede percibir la acción del usuario y reaccionar de acuerdo al comportamiento establecido por un **objeto oyente**.
- Algunos de los atributos son **texto**, **imagen**, **alineación** del texto y de la imagen, **color**, **borde**, **letra mnemónica** y **forma**, además, puede estar habilitado o no.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

14

BOTONES

- Desarrolle una aplicación que permita establecer el color de fondo de una ventana. El color será rojo o verde de acuerdo al botón que se apriete.
- La ventana inicialmente debe aparecer así:



- Éste es el aspecto cuando se aprieta rojo o verde:



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

15

```
//importar paquetes
class GuiFondoRojoVerde extends JFrame {
    /*Declarar los BOTONES*/
    public GuiFondoRojoVerde () {
        /*Establecer los valores de los atributos del frame*/
        /*Crear los BOTONES*/
        /*Crear y registrar los oyentes para los BOTONES*/
        /*Insertar los BOTONES en el panel de contenido*/
    }
    /*definir clases para establecer el comportamiento de los BOTONES*/
}
```

La estructura del código favorece la legibilidad

```
0
1
0
0
0
0
0
0
0
```

16

BOTONES

```
//importar paquetes
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

- Java brinda muchas facilidades a través de paquetes.
- El programador importa solo los paquetes que necesita.
- El tamaño del código que se genera al compilar depende solo de las facilidades básicas y los paquetes importados.

```
0
1
0
0
0
0
0
0
0
```

17

BOTONES

```
//importar paquetes
class GuiFondoRojoVerde extends JFrame {
    ...
}
```

- Un objeto de clase **GuiFondoRojoVerde** es también una instancia de **JFrame**.
- Como todo frame un objeto de clase **GuiFondoRojoVerde** tiene como atributos el borde, la barra de título y un panel de contenido.

```
0
1
0
0
0
```

18

BOTONES

```
//importar paquetes
...
class GuiFondoRojoVerde extends JFrame {
    /*Declarar los botones*/
    private JButton botonRojo, botonVerde;
}
...
```

- Los atributos de instancia de la clase `GuiFondoRojoVerde` son dos variables de la clase `JButton` provista por Swing.

19

BOTONES

```
class GuiFondoRojoVerde extends JFrame {
    /*Declarar los botones*/
    ...
    public GuiFondoRojoVerde () {
        /*Establecer los valores de los atributos*/
        setLayout(new FlowLayout());
        setSize(200, 120);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    ...
}
```

- La clase `JFrame` brinda servicios que permiten establecer los valores de los atributos del frame, por ejemplo el diagramado y el tamaño.

20

BOTONES

```
class GuiFondoRojoVerde extends JFrame {
    /*Declarar los botones*/
    ...
    public GuiFondoRojoVerde () {
        /*Establecer los valores de los atributos*/
        ...
        /*Crear los botones*/
        botonRojo = new JButton("Rojo");
        botonVerde = new JButton("Verde");
    }
    ...
}
```

- Por cada atributo de instancia que corresponde a una componente gráfica de la GUI, creamos un objeto gráfico.

21

BOTONES

```
class GuiFondoRojoVerde extends JFrame {
    ...
    public GuiFondoRojoVerde () {
        /*Establecer los valores de los atributos*/
        ...
        /*Crear los botones*/
        botonRojo = new JButton("Rojo");
        botonVerde = new JButton("Verde");
        /*Crear y registrar los oyentes para los botones*/
        OyenteBotonR ponerRojo = new OyenteBotonR();
        botonRojo.addActionListener(ponerRojo);
    }
    ...
}
```

- Para que un botón sea reactivo debe registrarse a un objeto oyente.
- El programador implementa cada clase de un oyente de acuerdo al comportamiento que debe exhibir el botón.

22

BOTONES

```
public GuiFondoRojoVerde () {
    ...
    /*Crear los botones*/
    botonRojo = new JButton("Rojo");
    botonVerde = new JButton("Verde");
    /*Crear y registrar los oyentes para los botones*/
    OyenteBotonR ponerRojo = new OyenteBotonR();
    botonRojo.addActionListener(ponerRojo);
    OyenteBotonV ponerVerde = new OyenteBotonV();
    botonVerde.addActionListener(ponerVerde);
    ...
}
```

23

BOTONES

```
public GuiFondoRojoVerde () {
    ...
    /*Insertar los botones en el panel de contenido*/
    getContentPane().add(botonRojo);
    getContentPane().add(botonVerde);
}
...
```

- Para que los botones sean visibles deben insertarse en un contenedor, en este caso el panel de contenido del frame.

24

BOTONES

```
/*definir clases para establecer el comportamiento de los
botones*/
class OyenteBotonR implements ActionListener {
    public void actionPerformed (ActionEvent event){
        getContentPane().setBackground(Color.red);
    }
}
```

- Para que un botón reaccione ante una acción del usuario, debe estar registrado a una clase que implemente a la **interface ActionListener**.
- **Implementar una interface** implica definir sus métodos.
- En este caso el método **actionPerformed** que recibe como parámetro un objeto de clase **ActionEvent**.

25

BOTONES

```
/*definir clases para establecer el comportamiento de los botones
class OyenteBotonR implements ActionListener {
    public void actionPerformed (ActionEvent event){
        getContentPane().setBackground(Color.red);
    }
}
class OyenteBotonV implements ActionListener {
    public void actionPerformed (ActionEvent event){
        getContentPane().setBackground(Color.green);
    }
}
```

- En este ejemplo, cada oyente modifica el color del panel de contenido de acuerdo al botón que el usuario oprime.

26

BOTONES

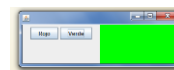
```
import java.awt.*;
class FondoBotones {
    public static void main(String args[] ) {

        GuiFondoRojoVerde f = new GuiFondoRojoVerde();
        f.setVisible(true);
    }
}
```

27

BOTONES

- Desarrolle una aplicación que permita establecer el color de un panel. El color será rojo o verde de acuerdo al botón que se apriete.
- Por ejemplo, cuando se oprima el botón verde la ventana debe aparecer:



28

```
//importar paquetes
class GuiPanelRojoVerde extends JFrame {
    /*Declarar los botones y los paneles */
    public GuiPanelRojoVerde () {
        /*Establecer los valores de los atributos */
        /*Crear los botones y los paneles*/
        /*Crear y registrar los oyentes para los botones*/
        /*Insertar los botones en el panel de botones y los paneles en
        el panel de contenido*/
    }
    /*definir clases para los oyentes*/
}
```

29

BOTONES

```
//importar paquetes
class GuiPanelRojoVerde extends JFrame {
    /*Declarar los botones y los paneles*/
    private JButton botonRojo, botonVerde;
    private JPanel panelColor, panelBotones;
}
```

- Los atributos de **GuiPanelRojoVerde** son dos botones y dos paneles.

30

BOTONES

```
public GuiPanelRojoVerde () {
    /*Establecer los valores de los atributos */

    /*Crear los botones y los paneles*/
    botonRojo = new JButton("Rojo");
    botonVerde = new JButton("Verde");
    panelColor = new JPanel();
    panelBotones = new JPanel();
    ...
}
```

- El orden de estas cuatro instrucciones es irrelevante en ejecución , pero afecta la legibilidad del código.

31

BOTONES

```
public GuiFondoRojoVerde () {
    ...
    /*Crear y registrar los oyentes para los botones*/
    OyenteBotonR ponerRojo = new OyenteBotonR();
    botonRojo.addActionListener(ponerRojo);
    OyenteBotonV ponerVerde =new OyenteBotonV();
    botonVerde.addActionListener(ponerVerde);
    ...
}
```

2

BOTONES

```
public GuiPanelRojoVerde (){\n    ...  
    /*Insertar los botones en el panel de botones y los paneles  
    en el panel de contenido*/  
    panelBotones.add(botonRojo);  
    panelBotones.add(botonVerde);  
    getContentPane().add(panelBotones);  
    getContentPane().add(panelColor);  
    ...  
}
```

- El orden de estas instrucciones influye en la apariencia de la GUI, si se invierten las dos últimas, cambia la distribución de los paneles, dado que por omisión se establece el diagramado FlowLayout.

33

BOTONES

```

/*definir clases para los oyentes*/
class OyenteBotonR implements ActionListener {
    public void actionPerformed(ActionEvent event){
        panelColor.setBackground(Color.red);
    }
}
class OyenteBotonV implements ActionListener {
    public void actionPerformed(ActionEvent event){
        panelColor.setBackground(Color.green);
    }
}
}

```

- La clase de cada oyente es **interna** a la clase que especializa a **JFrame** y por lo tanto tiene acceso a los atributos de instancia de **GuiPanelRojoVerde**, en particular a **panelColor**.

4

Programación basada en eventos

La **programación basada en eventos** es un modelo de programación en el cual el orden de la ejecución lo determina las acciones del usuario.



Programación basada en eventos

Un **evento** es una señal de que algo ha ocurrido.



Objetos y eventos

- Algunas componentes de una GUI son **reactivas**, pueden **percibir** las acciones del usuario y **reaccionar** en respuesta a ellas.
- Cuando un usuario realiza una acción sobre una componente reactiva se genera un **evento**.
- Un evento es una **señal** de que algo ha ocurrido.
- En esta materia consideraremos únicamente eventos generados por acciones del usuario al interactuar con la GUI.

```
01100
10011
10110
01110
01100
10011
10011
10110
01110
10011
1 11
0 0
1
```



Objetos y eventos

- Los **objetos fuente del evento**, están asociados a una componente de la interfaz, tienen una representación gráfica y son capaces de percibir y reaccionar ante un evento externo provocado por una acción del usuario y disparar eventos de software.
- Los **objetos evento** son disparados implícitamente por un objeto fuente del evento.
- Los **objetos oyentes** (listeners) se ejecutan para manejar un evento. La clase a la que pertenece un objeto oyente brinda métodos para manejar eventos, es decir especifica el curso de acción a seguir en respuesta a diferentes tipos de eventos.

```
01100
10011
10110
01110
10011
10011
10110
01110
10011
10011
1 11
0 0
1
```



Objetos y eventos

